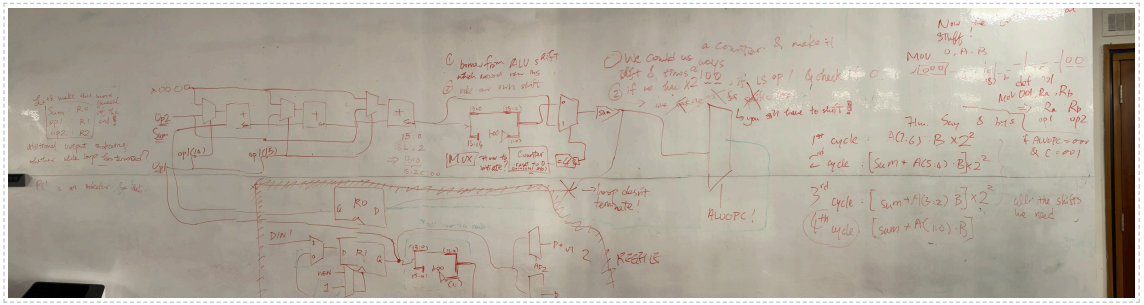
Challenge

Thursday, 13 March 2025 3:35 PM



We start off by trying to implement the equivalent software algorithm from 4 here, that will be the basic principle, then see how hardware could improve speed.

Methods we can use to reduce number of clock cycles:

- In software, each line of code corresponds to one assembly INS, which takes one clock cycle to execute, if we can replace that with *combinational logic*, the operation becomes instantaneous and saves a clock cycle
- Parallelism?
- A different algorithm for multiplication may lead to a faster result
- Potential software optimizations - e.g. 5x12 is faster than 12x5

```
0  // implement sum := op1 * op2 (LS 16 bits of)
1  // assume int = 16 bits (16 bit version of C).
2  unsigned int op1, op2, op2_shifted, sum; // variables
3  sum = 0; // initisalise
4  op2_shifted = op2; //initialise
5  while (op1 != 0) {
6      if (op1 & 1) { // & bitwise AND operation
7          sum = sum + op2_shifted;
8      }
```

```
 9       op2shifted = op2shifted << 1; // left shift by 1
10       op1 = op1 >> 1; // right shift by 1.
11   }
```

Figure 1: While loop implementation of multiply

Similar to the software algorithm, where we shift op1 and look bit-by-bit whether there is a term for $B*2^{n-1}$ then add to our cumulative sum, we try to do it as BASE 4 multiplication, i.e. shifting by 2 every time and looking at 2 bits altogher.

Say we have an 8-bit number 11100100 times another 8-bit number B:

$$11\ \ 10\ \ 01\ \ 00\ \ *B = \left(11_2 * B * 4^3\right) + \left(10_2 * B * 4^2\right) + \left(01_2 * B * 4^1\right) + \left(00_2 * B * 4^0\right)$$
$$= \left(3 * B * 2^6\right) + \left(2 * B * 2^4\right) + \left(1 * B * 2^2\right) + \left(0 * B * 2^0\right)$$

As we can see, there aren't too many combinations of multiplication, you either times 3, 2, 1, or 0

We can implement them using combinational addition all at once so that it only takes one clock cycle, by using *just 3 FULL ADDERS*, so we have even saved ourselves one FA in this implementation.

Moreover, note how we need an additional register to store op2_shifted, it shifts it by each time, going from LSB(2^0 weighting) to MSB(2^16 weighting) and adds appropriately to the Register containing the sum.
We can save this register as well by doing the addition from the MSB and SHIFTING THE SUM ITSELF TO THE LEFT

Consider A*B, if we did A(7:6)*B first then add to sum, and then left shift by 2, that is not enough yet, we would still have to shift twice
Now, we left shift A by 2, in principle that is equivalent to A(5:4)*B, so sum = (A(7:6)*B*2 + A(5:4))*2 = A(7:6)*B*2^2 + A(5:4)*2, notice how the weighting for A(7:6)*B has increased

Eventually, sum = $\{[(A(7:6) * B * 2 \quad + \quad A(5:4) * B) * 2 + A(3:2) * B] * 2\} + A(1:0)$
$= A(7:6) * B * 2^6 + A(5:4) * B * 2^4 + A(3:2) * B * 2^2 + A(1:0) * B * 2^0$
So it works, note however that we need to take special care at the very end not to do another left shift before switching SUM to the ALU main MUX to be written into a register

Couple of challenges to solve down the road:

- How do we make sure that the add-and-shift loop terminates? (PC = PC + 1)
- This takes multiple clock cycles to execute, if we define one instruction to do so, we need to modify the NEXT block so that it stalls PCNEXT suitably
- There is only one write port, yet we have to write SUM into our register AND AT THE SAME TIME shift op1 in the register file
- How much flexibility do we want with the operation? Do we want to fix sum at a certain Register, or op1 at a certain register??

There are too many degrees of freedom, so let's define our variables and registers first. I first thought it seemed straightforward to define R0 = sum, R1 = op1, R2 = op2shifted etc., however by considering the generality of an operation in a CPU, e.g. ADD Rc, Ra, Rb, it is more consistent if we do so as well.

We would like to define a new instruction MULTIPLY from MOV C_n, Ra, Rb (Ra := Ra op Rb),
where n = 1…7
Let's define MOV 1, RA, RB does RA*RB, and that RA = op1 and RB = op2
Which register to hold SUM? (refer to section on 1 write port, 2 writes)

The key to understanding this is that although it is a "new instruction that we can define", the way it is decoded is still according to a MOV instruction, so by default it is still going to MOV Rb into Ra, it STILL HAS A BASE TEMPLATE, which we can change of course; but, is that a necessary hassle?

The **control signals from DPDECODE** concerning this would be:

- WEN1 = 1
- A
- B
- C
- AD1SELC = 0 (recall Lab1, where we always write into Ra regardless of C field)

The **inputs into the REGFILE**:

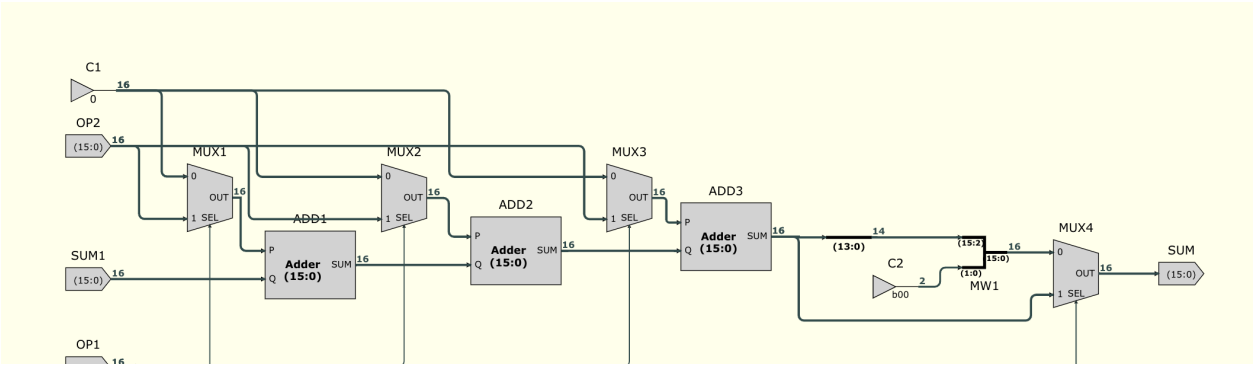
- WEN1
- DIN1 = RB
- AD1 = A
- AD2 is A but a DON'T CARE IN THE CURRENT CONTEXT (useful later)
- AD3 is B

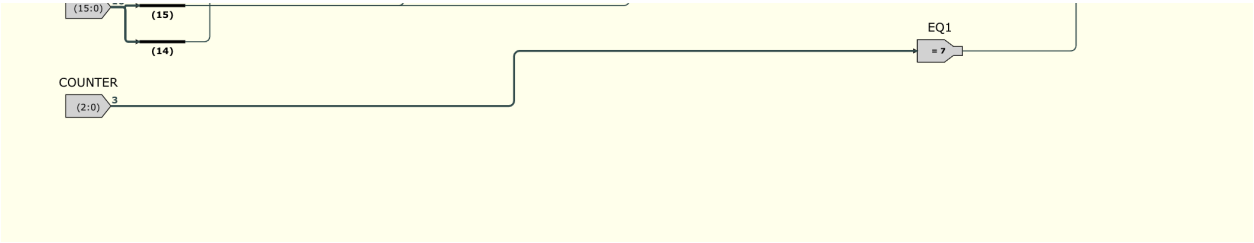
Outputs of REGFILE:

- DOUT2 = RA
- DOUT3 = RB

So we know that by our default template: we have RA, i.e. op1, and RB, i.e. op2 read from the register file and fed directly into the ALU for our use in multiplication!

Base 4 Add-and-shift





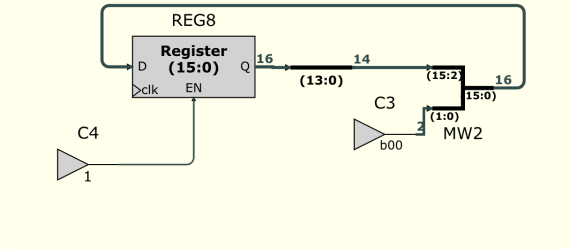
- We chain 3 adders together that add either 0 (X add) or Op2 to sum, selection implemented by 3 MUXes

- We look at the most significant 2 bits of op1, which is just A(15)*2 + A(14)*1, because of the doubled weighting for A(15), it is **selects 2 MUXes**
  Say A(15:14) = 11, sum = op2 + op2 + op2, but if A(15:14) = 01, sum = 0 + 0 + op2, we get 2 "op2"s fewer
- We have our own 2-bit shift before MUX4, notice ADD3.SUM has another wire bypassing the shift to MUX4, which addresses one point: we do not need another left shift after adding A(1:0)*B
- 3-bit counter: One issue of this design is since we are starting from the MSB and shifting one by one, we cannot stop the loops earlier and save clock cycles if op1 is already 0, we still need to shift to get the right weightings

  As a result however, this means all multiplications MUST take exactly 16/2 = 8 clock cycles to finish, so we could just use a counter that starts counting from 0 when the CPU is doing a MULT operation, and bypasses the shift when it has counted to the end, i.e. when COUNTER.Q = 111, once we move onto the next instruction, the counter should + 1 once more, making it RESET to 000 for future MULT operations

One write port, yet we have SUM AND OP1 to write to?

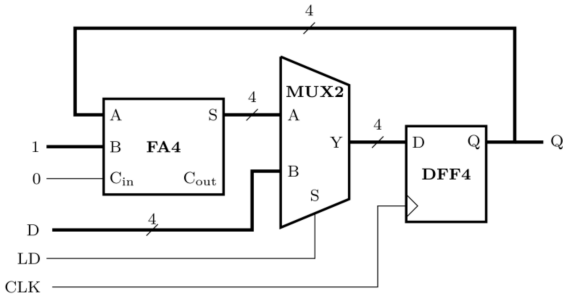
With help from Dr Ed Stott, it was suggested that one could define the register holding op1 to self-shift on each clock tick, thus we can use the write port on SUM:



Challenges:

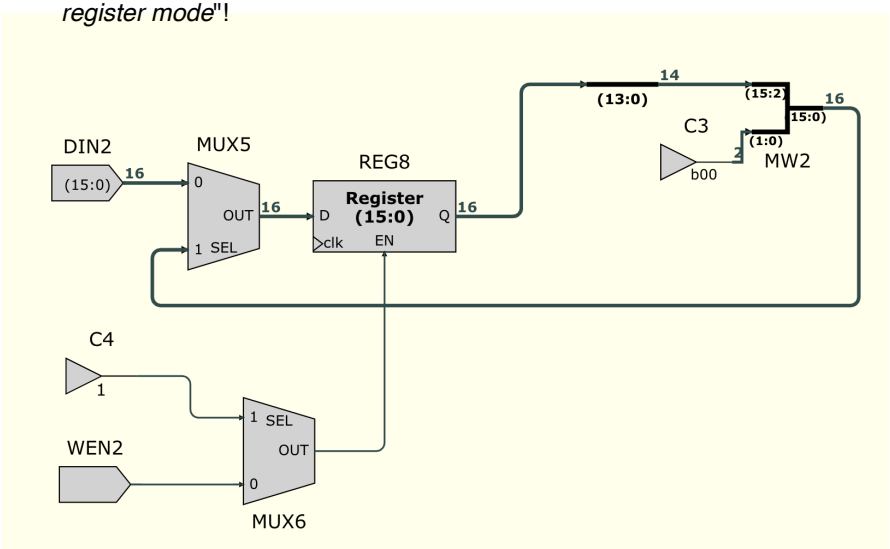
- What about initiating the Register value, we have no control over this shifter right now
- Say we do that for REG0, does that mean we can no longer do any default operations on REG0?
- Also, does that mean we have fixed op1 to be at R0, would that be a bit too inflexible? What if I want to do MOV 1, ***R3***, R5? Could I generalize this

The first challenge is related to Autumn Term labs:



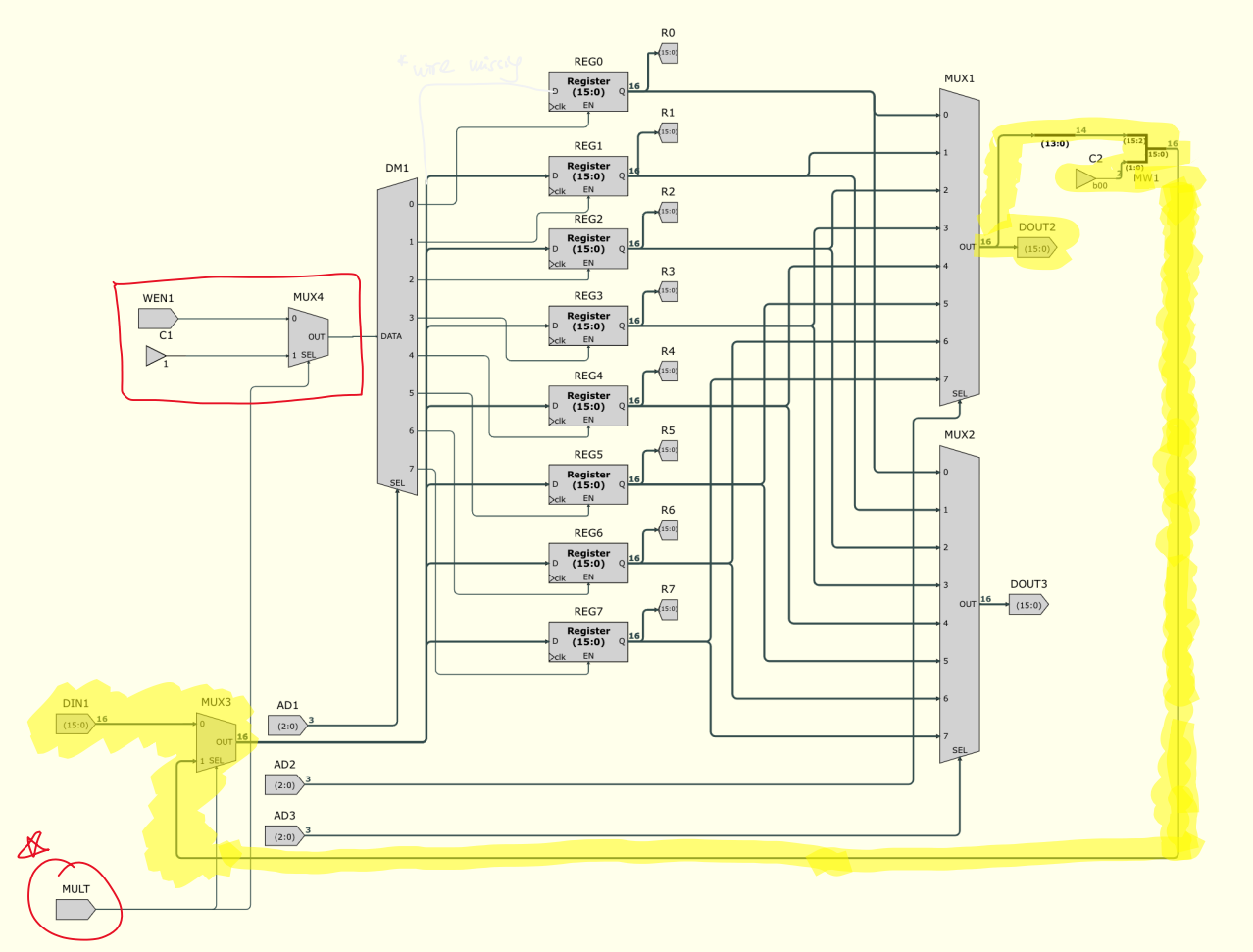
We use a MUX to ***break the feedback loop***

- I realized we initialize R0 just as you would do MOV R0, X as usual, so we still need the DIN and WEN connections
- In doing that, this simultaneously solves our 2nd problem - we can use MUX to switch REG0 between "*multiplication mode*" and "*normal register mode*"!



- When we are not multiplying, we want to switch DIN and WEN through, and when we are, we switch the other inputs that correspond to the multiplication implementation --> we can still perform operations other than multiplication on REG0!

As for whether we must set R0 to be op1, one might expect that for such flexibility, we would need to repeat the above for each register, but we can get around that by changing where we connect the MUXes to:



- This works in allowing op1 to be at any register, however in this case, the write port has been used, and we'd have to find another way to

write back to SUM, although this defies our original intentions, let's compare solutions:

- If NOT using a write port, we can still connect the register output directly to the corr. Input of the ALU, but the problem is we lose the ability of addressing, i.e. using a fixed register location, making the operation inflexible

- Write port on SUM REG --> Flexibile SUM register, but then op1 is fixed
- Write port on op1 --> Flexible op1 register, but SUM is fixed

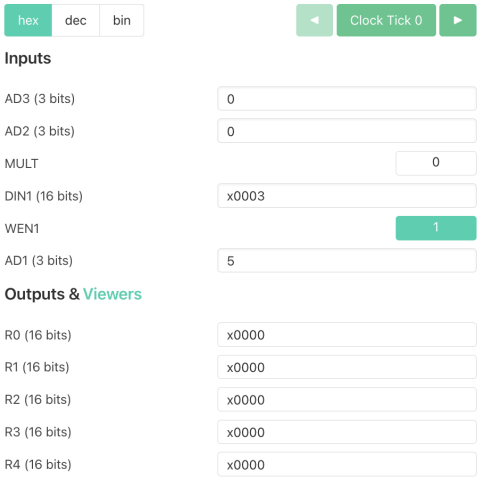
Either way one variable will be fixed at a certain register unless we add another write port, which would require new INS definitions (definitely not doing that)

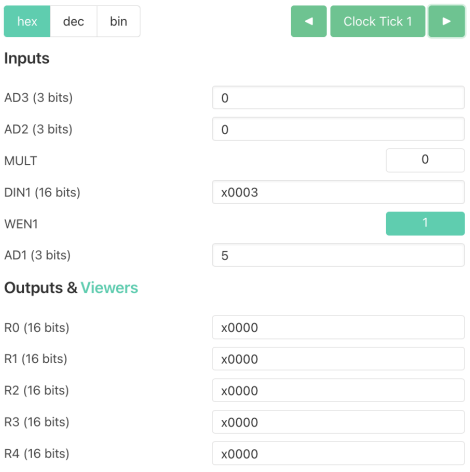
It makes more sense, however, that if you can choose where to store your op2, you should be able to do the same as well for op1. In this case we choose to fix SUM at REG0
***Fixing SUM to be at REG0 also means op1 can only be from R1 to R7

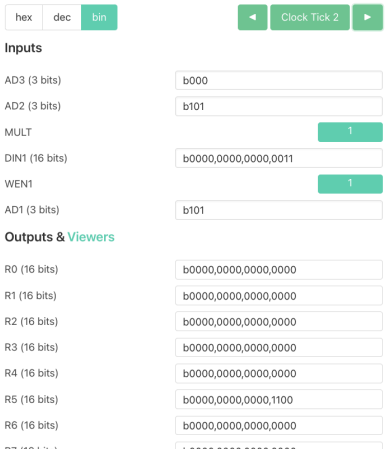
R0 and SUM

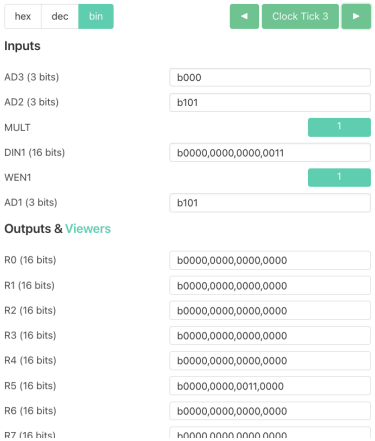
- Again, we will use a MUX to separate between "*multiplication mode*" and "*normal register mode*"
- Normal register mode, we switch through DIN1
- Multiplication mode, we switch through the SUM output from ALU

Testing the flexible op1 Register shifting:









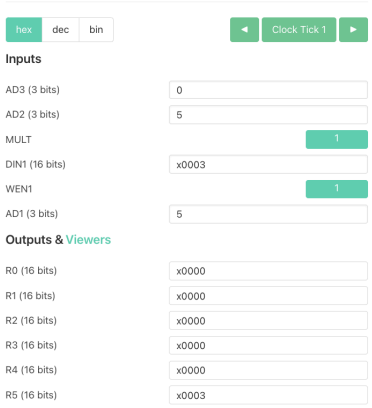
R5 (16 bits) x0000

- MULT = 0 here, we first initialize R5 with the value b0000,0000,0000,0011

R5 (16 bits) x0003

Say after the first clock tick, we encounter a MULT ins where R5 is op1

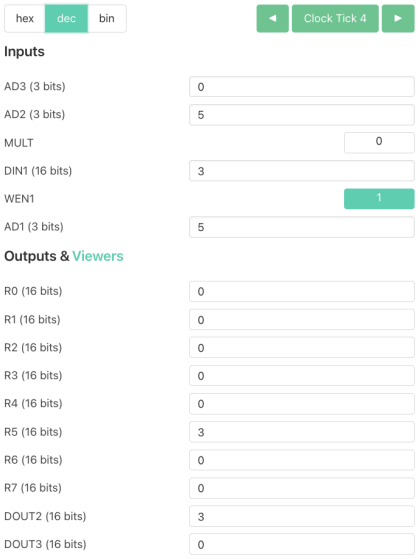
- MOV 1, R5, XX
- Note that because of how the MOV ins is decoded, **AD1 = Ra = AD2** = 5
- We set MULT to 1, AD2 = AD1 = 5
-



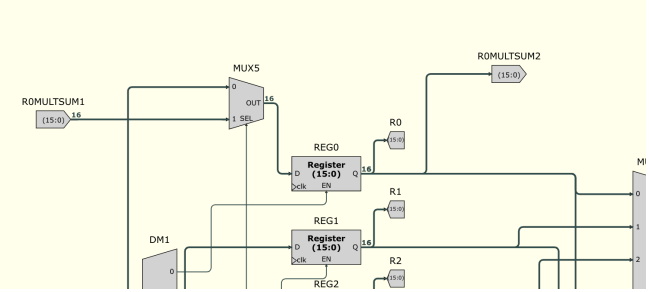
DOUT2 (16 bits) b0000,0000,0000,1100

DOUT2 (16 bits) b0000,0000,0011,0000

- We can see that R5 and DOUT2 are shifting to the left by 2 bits after each tick
- Say, now MULT = 0, we expect after the next clock tick, since AD1 is still 5, WEN1 1 and DIN1 still 3, R5 and DOUT2 will return to 3



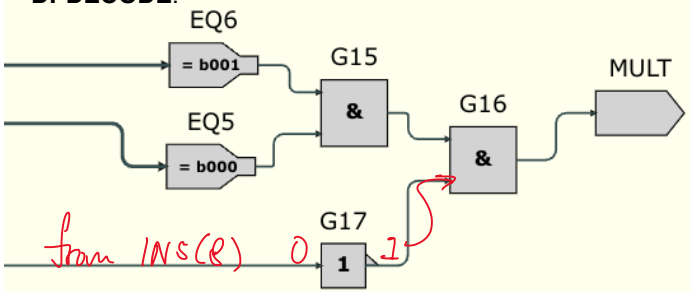
- Checks out!



MULT Control Signal

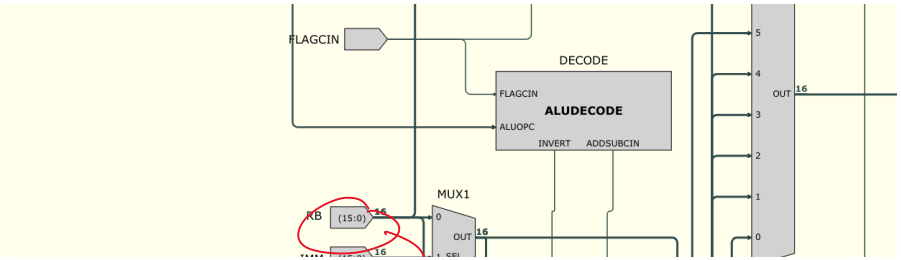
- ***NOTE***: I have been saying which inputs to switch through **when we are/are not multiplying**, but what would be the select input to both MUXes?

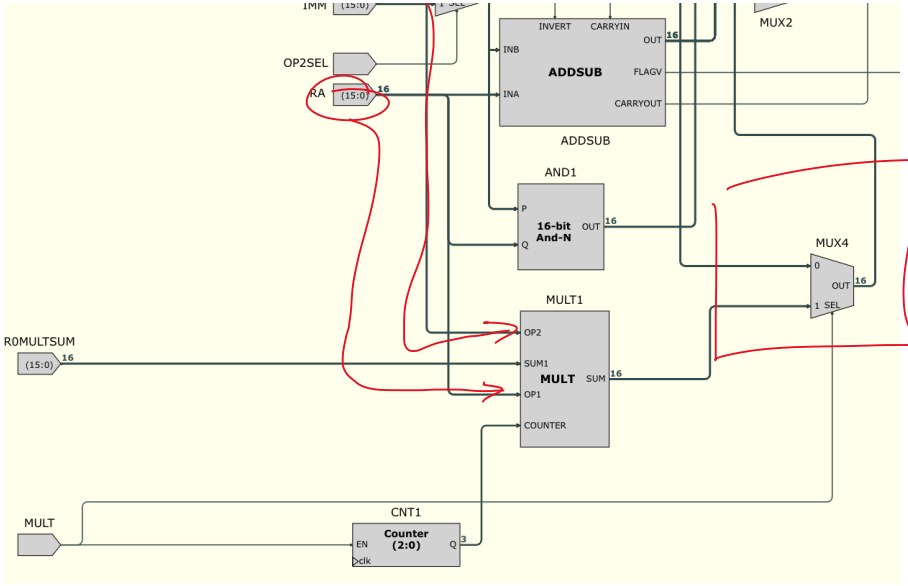
- Since we want to tell if we are doing a MULT operation or not, that depends on the INS itself, which is why here we have connected a MULT input, which will be a control signal from our **MODIFIED DPDECODE**.



DPDECODE: Control signal for whether we are doing a MULT (when it is a MOV Rc, Ra, Rb ins where C = 001)

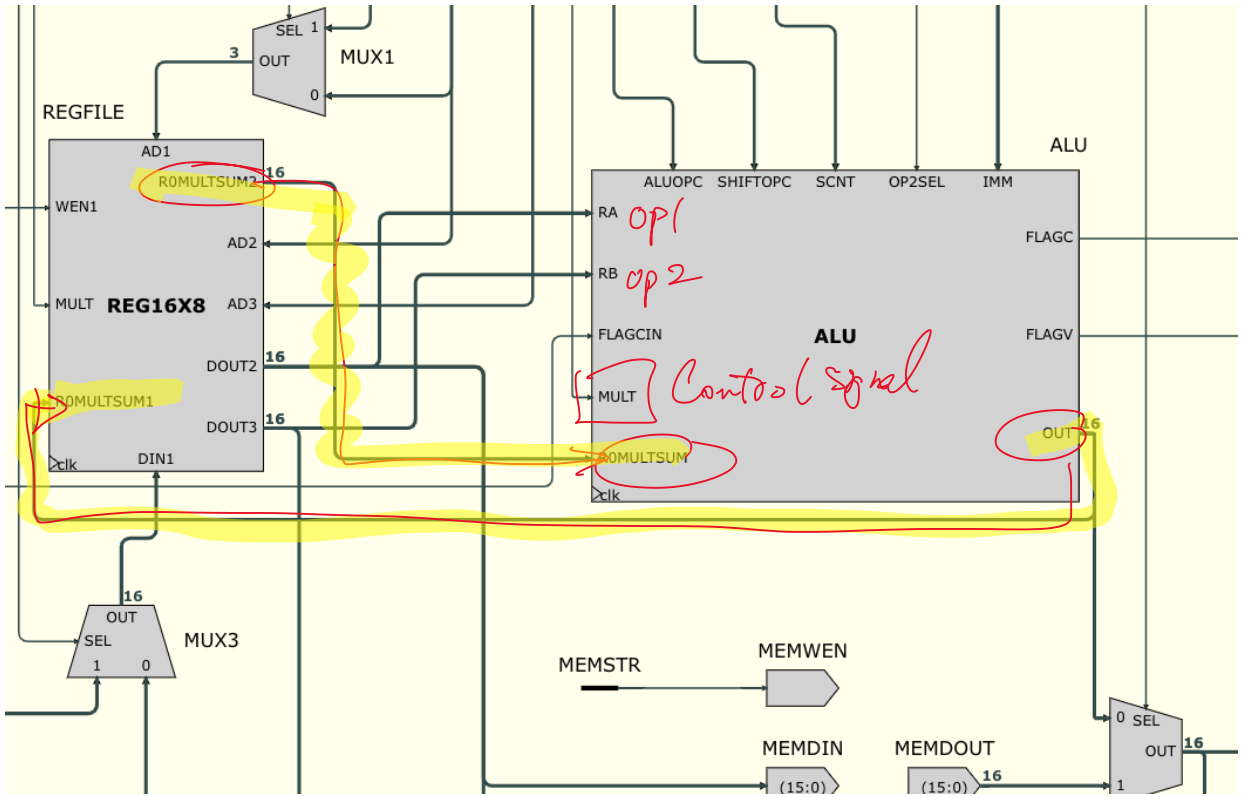
- NEED TO CHECK IF IT IS MOV Ra, IMMS8! -> we want INS(8) to be 0
- Set up similarly to CONTROLDECODE for ins.JUMP





We need additional logic so that when ALUOPC = 000, i.e. MOV, the correct MOV operation results appears at MUX.0

When MULT = 1, switch MULT.SUM through instead of Rb/Imm



Remaining Challenge: Counter and PC Stall

It would be helpful to draw some timelines, make sure the counters aren't miscounting / one tick delay etc.

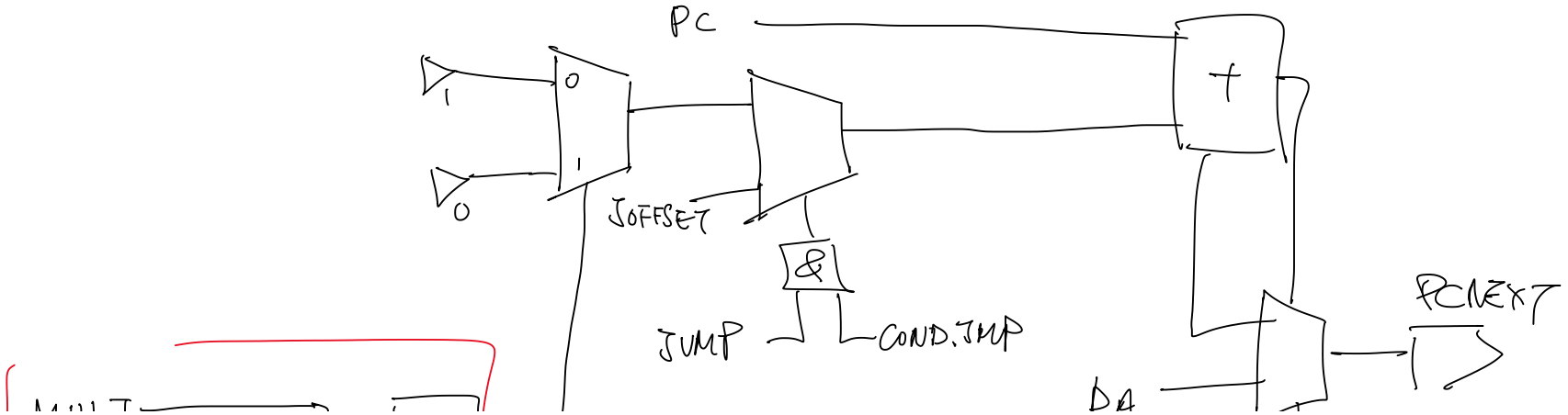
Say we have a MULT ins at PC = 0x0001 where:
Op1 = A = 0000 0000 1011 0011
Op2 = B = 0000 0000 0110 0101

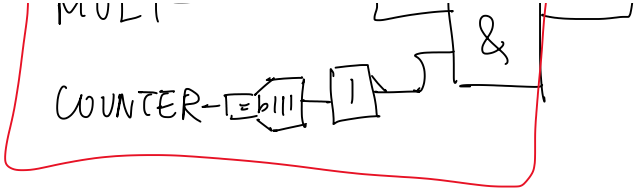
Coloured in Blue is the desired operation

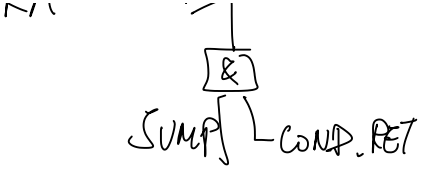
Clock Cycle	0	1	2	3	4	5	6	7	8	9
PC.NEXT = PC.D	1	1	1	1	1	1	1	1	2	3
PC.Q	0	1	1	1	1	1	1	1	1	2
R0.D = ALU.OUT	0	S = A(15:14)*2	S+A(13:12)B]*2	S+A(11:10)B]*2	S+A(9:8)B]*2	S+A(7:6)B]*2	S+A(5:4)B]*2	S+A(3:2)B]*2	S+A(1:0)B	/
R0.Q	0	/								
Counter	0	0	1	2	3	4	5	6	7	***0***
MULT	0	1	1	1	1	1	1	1	1	0

- After step simulation in the ALU sheet, it can be seen that Counter value does reset to 0 at clock tick 9 and does not increment anymore when MULT = 0
- So the default counter does work as intended!

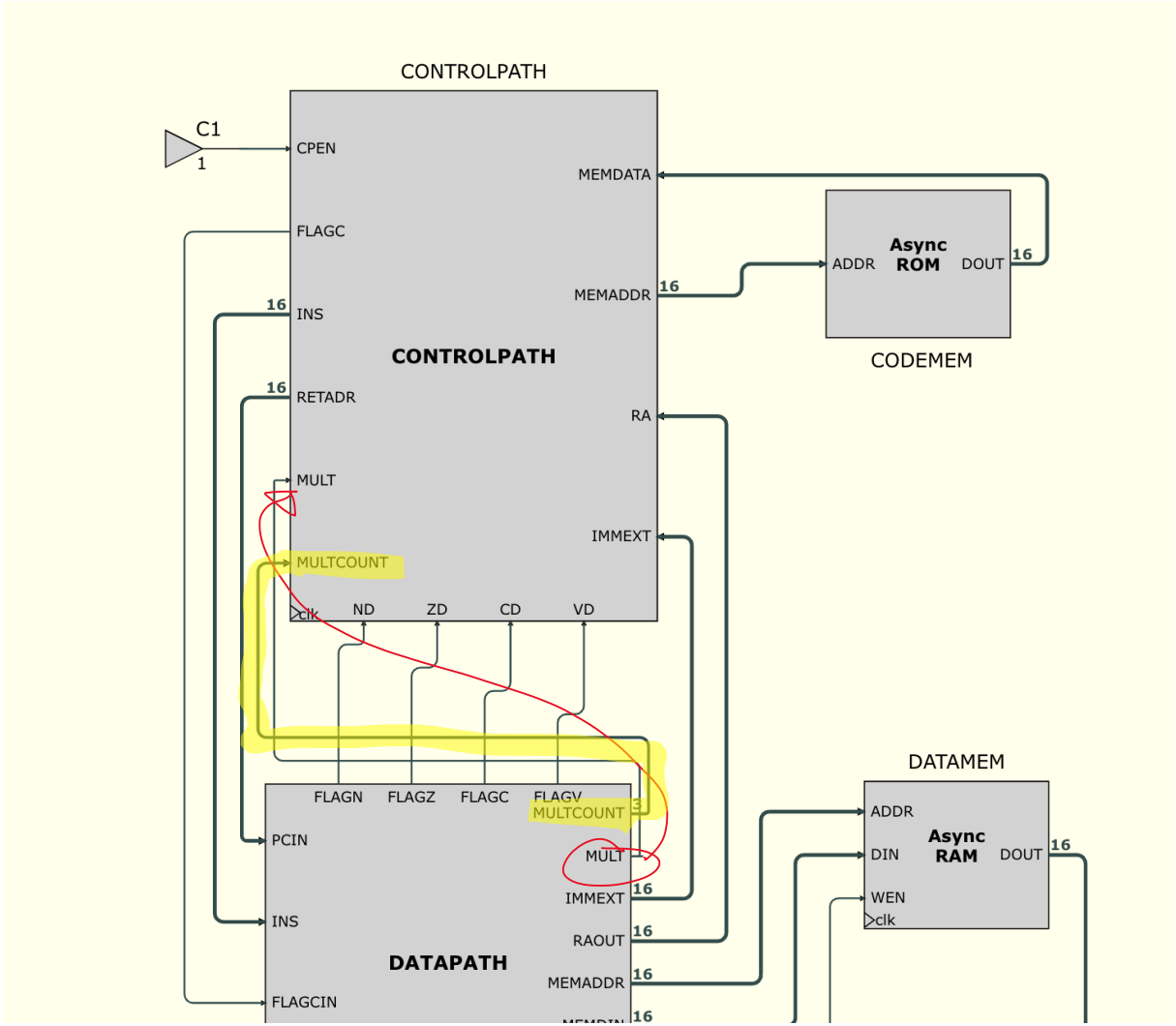
Now we look at how to implement PC stall:
- When MULT = 1, we hold PC.NEXT, i.e. PC.NEXT = PC + 0
- When Counter = 7, PC.NEXT = PC + 1

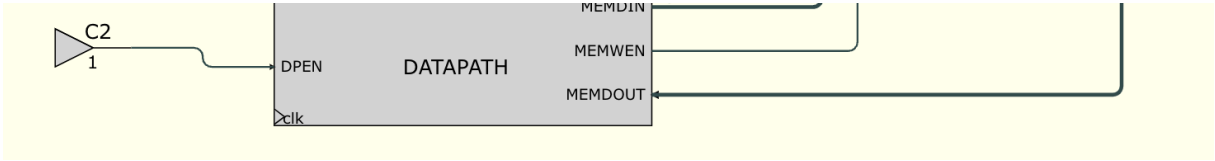






- Added corresponding inputs and outputs so that COUNTER value could be fed into NEXT block of CONTROLPATH

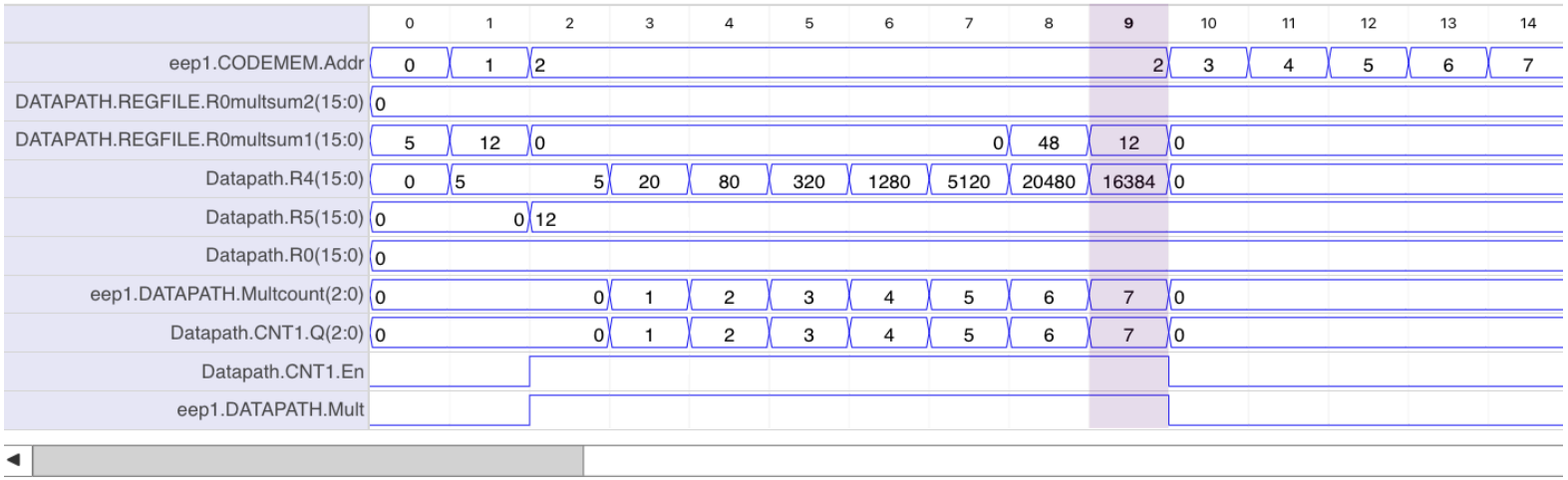





```
1    MOV R4, #5
2    MOV R5, #12
3    MOVC1, R4, R5 // Expect to get R0 = 60
```

```
1    0x00 0x0905
2    0x01 0x0b0c
3    0x02 0x08a4
4
```

NOT WORKING

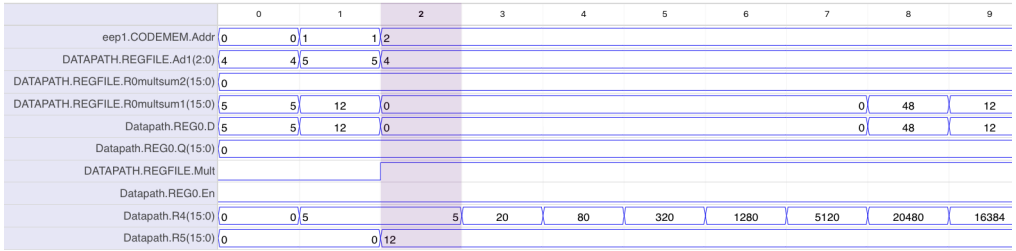


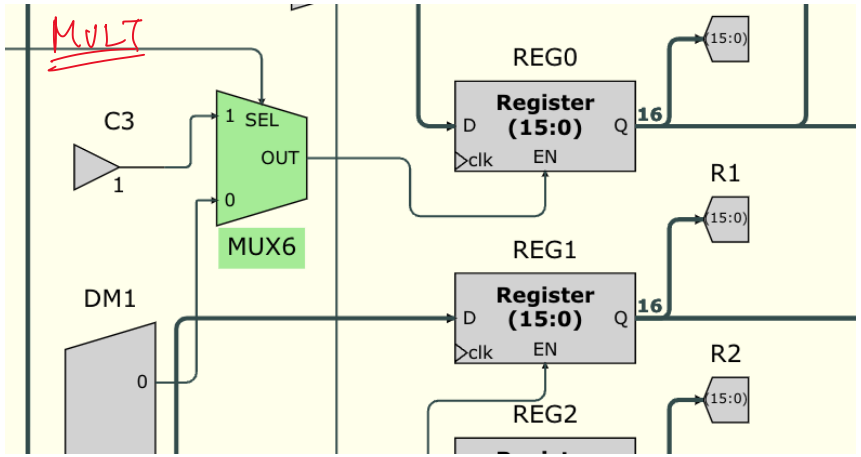
We can see CNT1.EN, MULTCOUNT, PC value all operate as expected
and at the right clock cycles.

We also see R4, i.e. op1 shift left as expected
R5 is fine
However, R0multsum2 stays at 0, R0 stays at 0

Upon further inspection around R0, it can be seen that REG0.EN is at 0 the

entire time --> enable logic has not been updated to include "multiplication mode" and "normal mode"!
REG0.EN currently depends on AD1 switching a 1 through the DEMUX --> AD1 corresponds to op1 which is anything but 0!

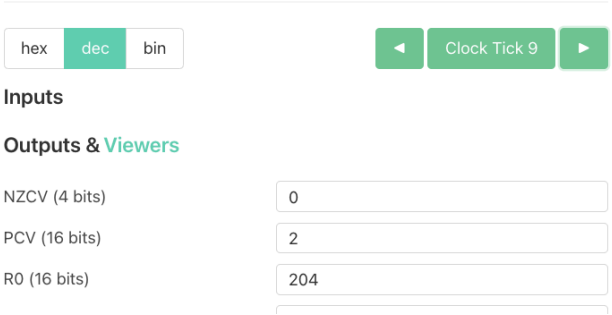




After trouble-shooting:

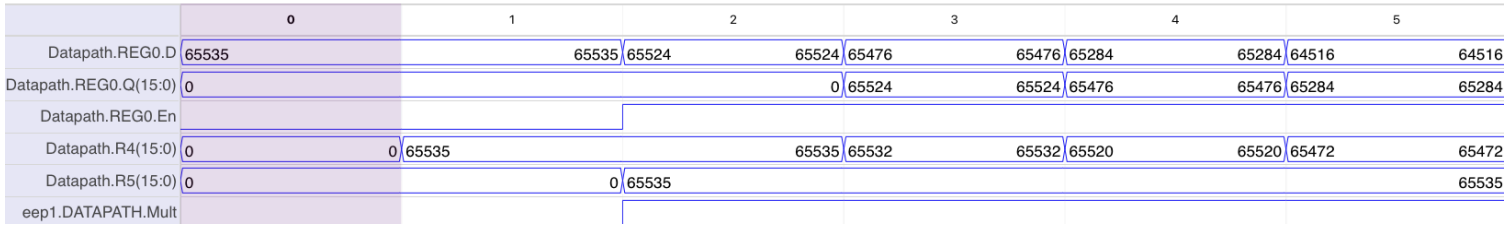
- One tick-delay between calculating the result and it appearing at REG0, as expected
- Exactly 8 clock ticks
- We indeed get 60 at REG0

Testing with other values:
12x17 = 204



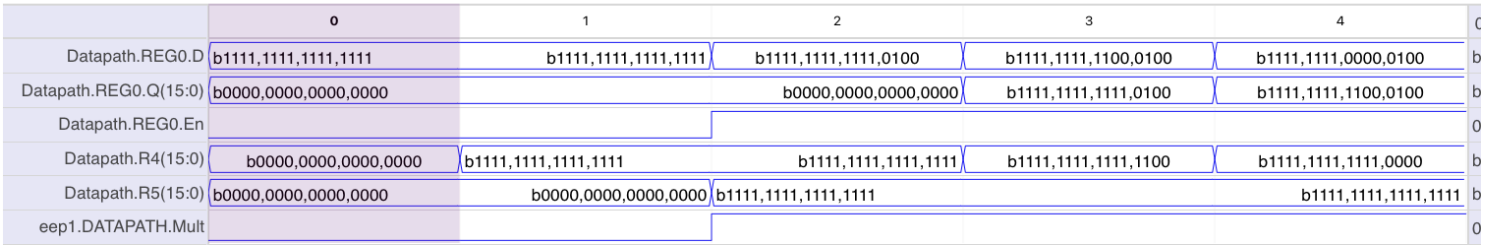
127x126 = 16,002

255x255 = 65,025



- REG0 value gets smaller an
- d smaller

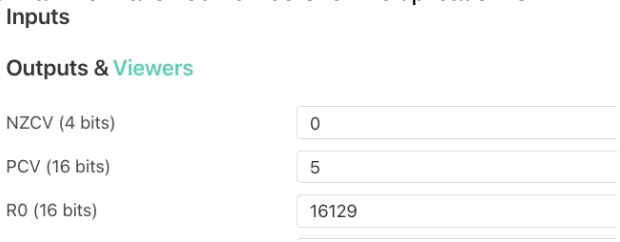
Look at it in binary:



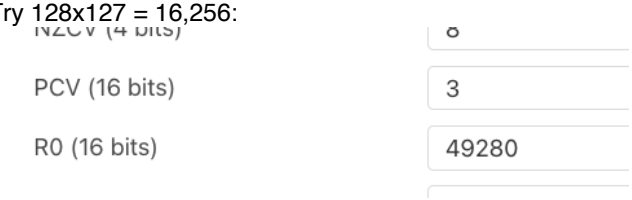
- We intend to multiply 0000 0000 1111 1111 with 0000 0000 1111 1111, but recall that DPDECODE sign extends our 8 bit immediate before MOV into a register.
- 255 has 1 at MSB of our IMMS8, which is a -ve number when interpreted as sign -> different value after sign extended to a 16-bit word

- **We find one unexpected limitation of the multiplier - it only allows multiplication of 7-bit unsigned numbers**

So Maximum allowed numbers for multiplication is: 127 x 127 = 16,129



Try 128x127 = 16,256:



We do not get the correct product.

Evaluation

Advantages:

- Used only 3 FA
- Exactly 8 clock cycles, so there is no ambiguity in whether the value has been computed or not (e.g. if you had 0x0 but the clock cycle needed is unknown, you can never be 100% sure if we have exited the add-and-shift loop
- There is flexibility in register location for both op1 and op2
- Registers have a "multiplication mode" and "normal mode", we don't lose the default functionality of a particular register because of the multiplication operation
- Starting from the most significant 2 bits saves one register during the MULT operation, we DO NOT need an op2_shifted

Disadvantages:

- Although starting from the most significant 2 bits saves one register, as a result we cannot end the loop earlier even if op1 is already x0000 before 8 shifts are completed
- Could be even faster
- Quite a lot of MUXes used

Possible Improvements

- Using pipelining to improve speed
- Have a special "unsign extended" version of op1 and op2 which would allow 8 bit multiplication
- To implement 16 bit x 16 bit, we need 32 bits to cover all products, or 2 16-bit registers, in which case the above shift-by-2 method is not ideal (we cannot shift in a carry for left shifts, and since we are shifting by 2 at once, that means doing 4 ADD/ADC instructions, quite inefficient). We could potentially get around that using a 32-bit temporary register for the shift operations, and then use a bus spreader to write into the registers

Record of proof that the Challenge was done in Labs